

Doris IVF 向量检索召回率测试（源码核对修正版）

基于 ANN / IVF 索引的 TopK 向量检索 Recall@K 验证 · 经 Doris 源码核对

核心结论：Ground Truth 用精确 `l2_distance`；ANN 侧用近似 `l2_distance_approximate`，且必须是逐条 query 的 `ORDER BY ... LIMIT k` 直查。只有这种查询形状才会命中 IVF 下推规则（`PushDownVectorTopNIntoOlapScan.java`）。用错函数或用 JOIN+窗口写法，都会绕过 IVF，使 Recall 恒为 1.0，测试失效。

0. 测试原理

IVF 是近似最近邻检索。测召回率需先用精确暴力搜索得到 Ground Truth，再与 IVF ANN 结果做 ID 交集统计：

$$\text{Recall@K} = |\text{ANN_TopK} \cap \text{Exact_TopK}| / K$$

示例：

Exact Top10 = [1,2,3,4,5,6,7,8,9,10]

ANN Top10 = [1,2,3,4,5,6,7,8,20,30]

Recall@10 = 8 / 10 = 0.8

- Exact TopK：必须用精确函数 `l2_distance`，不走 ANN 索引。
- ANN TopK：必须用 `l2_distance_approximate` 并经 EXPLAIN 确认走了 IVF。
- 两边必须使用同一 query 向量、同一距离类型、同一过滤条件、同一 TopK。

1. 建表（IVF 表 + 查询向量表）

精确函数本就不走索引，因此 Ground Truth 无需单独建无索引表，直接在 IVF 表上用 `l2_distance` 即可。示例按 128 维、L2 距离、nlist=1024 编写，维度/桶数/副本数按实际集群调整。

```

-- 数据表（带 IVF 索引）
CREATE TABLE vec_ivf (
    id BIGINT NOT NULL,
    vec ARRAY<FLOAT> NOT NULL,
    INDEX ann_idx_vec (vec) USING ANN PROPERTIES(
        "index_type" = "ivf",
        "metric_type" = "l2_distance",
        "dim" = "128",
        "nlist" = "1024"
    )
) ENGINE=OLAP DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 32
PROPERTIES ("replication_num" = "1");

-- 查询向量表
CREATE TABLE query_vec (
    qid BIGINT NOT NULL,
    qvec ARRAY<FLOAT> NOT NULL
) ENGINE=OLAP DUPLICATE KEY(qid)
DISTRIBUTED BY HASH(qid) BUCKETS 1
PROPERTIES ("replication_num" = "1");

```

2. 先验证「ANN 查询确实走了 IVF」（最关键）

任取一条 query 向量，对直查写法做 EXPLAIN，计划里应出现虚拟列 / ANN TopN 下推：

```

EXPLAIN
SELECT id
FROM vec_ivf
ORDER BY l2_distance_approximate(vec, [0.1, 0.2, /* ...共128维 */]) ASC
LIMIT 10;

```

对比：把 `l2_distance_approximate` 换成 `l2_distance`，EXPLAIN 里就不会有 ANN 下推。建议保留两份 EXPLAIN 文本作为测试报告附件。

3. 生成 Ground Truth（精确，可批量）

精确侧不走索引，可用 JOIN + 窗口一次性批量算并落表：

```

CREATE TABLE gt_top10 (
    qid BIGINT, id BIGINT, rn BIGINT
) ENGINE=OLAP DUPLICATE KEY(qid, id)
DISTRIBUTED BY HASH(qid) BUCKETS 8
PROPERTIES ("replication_num" = "1");

INSERT INTO gt_top10
SELECT qid, id, rn FROM (
    SELECT q.qid, t.id,
        ROW_NUMBER() OVER (
            PARTITION BY q.qid
            ORDER BY l2_distance(t.vec, q.qvec) ASC, t.id ASC    -- 精确，不走索引
        ) AS rn
    FROM query_vec q JOIN vec_ivf t
) x
WHERE rn <= 10;

```

4. 生成 IVF ANN TopK (近似，必须逐条 query)

不能用 JOIN 批量——JOIN+窗口的形状下推不了。下推规则要求的计划形状是 `LogicalTopN → Project → (Filter) → OlapScan`，即直接对带索引的表做 `ORDER BY ... LIMIT k`。建好收集表：

```

CREATE TABLE ann_top10_nprobe32 (
    qid BIGINT, id BIGINT
) ENGINE=OLAP DUPLICATE KEY(qid, id)
DISTRIBUTED BY HASH(qid) BUCKETS 8
PROPERTIES ("replication_num" = "1");

```

对每一条 query (qid 固定、向量字面量内联) 执行：

```

SET ivf_nprobe = 32;          -- 会话变量，源码 vector_search_user_params.h 默认 32

INSERT INTO ann_top10_nprobe32
SELECT 1 AS qid, id          -- qid 换成当前这条的值
FROM vec_ivf
ORDER BY l2_distance_approximate(vec, [0.1, 0.2, /* ...128维字面量 */]) ASC
LIMIT 10;

```

两个硬性要求：

- 向量必须是字面量数组内联进 SQL，不能写成 `q.qvec` 子查询 / JOIN，否则下推失效。
- 一条 query 一条 `INSERT ... SELECT`，用脚本循环全部 query。

最小可用的循环脚本 (bash + mysql client)：

```
# 假设把 query 向量导出成: qid<TAB>[0.1,0.2,...]
NPROBE=32
while IFS=$'\t' read -r qid qvec; do
    mysql -h$FE -P9030 -uroot db -e "
        SET ivf_nprobe=${NPROBE};
        INSERT INTO ann_top10_nprobe${NPROBE}
        SELECT ${qid} AS qid, id
        FROM vec_ivf
        ORDER BY l2_distance_approximate(vec, ${qvec}) ASC
        LIMIT 10;"
done < queries.tsv
```

5. 计算 Recall@10

按 query 求交集占比，再对所有 query 求平均：

```
SELECT AVG(recall) AS recall_at_10
FROM (
    SELECT g.qid, COUNT(a.id) / 10.0 AS recall
    FROM gt_top10 g
    LEFT JOIN ann_top10_nprobe32 a
        ON g.qid = a.qid AND g.id = a.id
    GROUP BY g.qid
) r;
```

定位低召回的 query：

```
SELECT g.qid, COUNT(a.id) AS hit_count, COUNT(a.id)/10.0 AS recall_at_10
FROM gt_top10 g
LEFT JOIN ann_top10_nprobe32 a
    ON g.qid = a.qid AND g.id = a.id
GROUP BY g.qid
ORDER BY recall_at_10 ASC;
```

6. 扫不同 nprobe 对比

GT 只生成一次。换 `nprobe` 时改 `SET ivf_nprobe=N` 并落到对应的 `ann_top10_nprobeN` 表，重复第 4、5 步。建议测 8 / 16 / 32 / 64 / 128：

```

SELECT 'nprobe=8' AS param, AVG(recall) FROM (
    SELECT g.qid, COUNT(a.id)/10.0 AS recall FROM gt_top10 g
    LEFT JOIN ann_top10_nprobe8 a ON g.qid=a.qid AND g.id=a.id GROUP BY g.qid) r
UNION ALL
SELECT 'nprobe=32' AS param, AVG(recall) FROM (
    SELECT g.qid, COUNT(a.id)/10.0 AS recall FROM gt_top10 g
    LEFT JOIN ann_top10_nprobe32 a ON g.qid=a.qid AND g.id=a.id GROUP BY g.qid) r
ORDER BY param;

```

7. 建议报告输出格式

不要只输出 Recall，应同时记录延迟和 QPS，否则无法判断参数是否可用于生产。

index_type	topK	nprobe	query_count	avg_latency_ms	p99_latency_ms	recall_at_10
IVF	10	8	1000	-	-	-
IVF	10	16	1000	-	-	-
IVF	10	32	1000	-	-	-
IVF	10	64	1000	-	-	-
IVF	10	128	1000	-	-	-

8. 关键检查清单（踩中任一条结果即失效）

- ANN 侧用 `l2_distance_approximate`，GT 侧用 `l2_distance`（精确）。两个函数搞反 = 测不出东西。
- ANN 侧是逐条 query 的 `ORDER BY ... approximate ... LIMIT k`，不是 JOIN+窗口。EXPLAIN 必须看到 ANN 下推。
- 两侧 query 向量、距离类型（l2 / ip）、`LIMIT k`、`WHERE` 过滤条件完全一致。
- `metric_type` 与业务语义一致：cosine 需先归一化向量，再用 `inner_product`。
- 改 `nprobe` 后重新生成 ANN 表，GT 表不用重算。
- 并发压测注意缓存影响：报告建议区分冷查询、预热后查询、并发查询。

9. 源码对应关系

关注点	源码 / 测试路径	含义
触发 IVF 下推	fe/.../rules/rewrite/ PushDownVectorTopNIntoOlapScan.java	仅当 ORDER BY 为 L2DistanceApproximate / InnerProductApproximate 且为 TopN→Project→Scan 形状时才下推到 ANN 索引。
近似函数定义	fe/.../scalar/L2DistanceApproximate.java、 InnerProductApproximate.java	l2_distance_approximate / inner_product_approximate；非确定性，触发 ANN 路径。
IVF 建表参数	regression-test/suites/ann_index_p0/ ivf_index_test.groovy	USING ANN、index_type=ivf、metric_type、dim、nlist 建表与 topn 查询示例。
查询参数	be/src/exec/scan/vector_search_user_params.h	ivf_nprobe（默认 32）、hnsf_ef_search 等会话级向量搜索参数。
IVF 搜索执行	be/src/storage/index/ann/faiss_ann_index.cpp	查询时把 ivf_nprobe 转换为 FAISS SearchParametersIVF.nprobe。
结果对比对象	be/src/storage/index/ann/ann_search_params.h	ANN TopN 输出 row_ids 与 distances，召回率测试比较 id 集合。

Doris IVF Recall Test · 源码核对修正版